

37. GPU 上位速度列探索

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	scripts/gpu_upper_policy_search.py
種別	Python
区分	planning research
役割	全レース distance-indexed speed policy を GPU 上で多段 coarse-to-fine に探索する。
起動文脈	本番前の warm start policy 生成側。

このファイルを読む理由

全レース distance-indexed speed policy を GPU 上で多段 coarse-to-fine に探索する。

- runtime MPC を置き換えるのではなく warm start policy 候補を作る。

呼び出し関係

どこから呼ばれるか

- GPUbatch/shellcampaign

次に読むと理解が進むファイル

- scripts/validate_gpu_upper_policy_candidates.py
- scripts/run_upper_mesh_convergence.py

同一リポジトリ内の主要依存

- mpc_solarcar/route_utils.py

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

GPU CEM proposer for a fine distance-domain upper speed policy.

This is deliberately a multi-fidelity proposer. Its vectorized energy model searches thousands of distance-indexed policies concurrently on CUDA. The selected policy must then be re-optimized and validated by “scripts/solar_sim.py”; the GPU surrogate is never treated as the production acceptance model.

代表コード断片

```
)

class TensorMap2D:
    """Bilinear CSV-map interpolation that stays on the selected torch device."""

    def __init__(self, csv_path: Path, device: torch.device):
        frame = pd.read_csv(csv_path, index_col=0)
        self.x = torch.as_tensor(frame.index.to_numpy(dtype=np.float32), device=device)
        self.y = torch.as_tensor(frame.columns.to_numpy(dtype=np.float32), device=device)
        self.z = torch.as_tensor(frame.to_numpy(dtype=np.float32), device=device)
        if len(self.x) < 2 or len(self.y) < 2 or self.z.shape != (len(self.x), len(self.y)):
            raise ValueError(f"invalid 2-D map: {csv_path}")

    def sample(self, x: torch.Tensor, y: torch.Tensor) -> torch.Tensor:
        x_value = torch.minimum(torch.maximum(x, self.x[0]), self.x[-1]).contiguous()
        y_value = torch.minimum(torch.maximum(y, self.y[0]), self.y[-1]).contiguous()
        i1 = torch.clamp(torch.searchsorted(self.x, x_value), 1, len(self.x) - 1)
        j1 = torch.clamp(torch.searchsorted(self.y, y_value), 1, len(self.y) - 1)
        i0 = i1 - 1
        j0 = j1 - 1
        wx = (x_value - self.x[i0]) / torch.clamp(self.x[i1] - self.x[i0], min=1.0e-9)
```

主要構造の読み取り

主要クラスは TensorMap2D, TensorMap1D, WeatherGrid。主要関数は resolve, iso_utc, source_signature, sample_cem_noise, cem_should_stop, resolve_cuda_graph_enabled, build_distance_segments, kinetic_power_w。CLI 引数宣言は 20 件。

主要クラス・関数

行	種別	名前	補足
208	class	TensorMap2D	
236	class	TensorMap1D	
252	class	WeatherGrid	
39	function	resolve	args: profile, value
44	function	iso_utc	args: value
50	function	source_signature	args: paths
62	function	sample_cem_noise	args: population, dimensions
85	function	cem_should_stop	
102	function	resolve_cuda_graph_enabled	args: requested, integration_ds_km
109	function	build_distance_segments	args: race_km, integration_ds_km, stop_distances_km
133	function	kinetic_power_w	args: mass_kg, speed_ms, previous_speed_ms, dt_sec
146	function	slew_limited_segment_kinematics	args: previous_speed_ms, target_speed_ms, distance_km
193	function	stationary_auxiliary_power_w	args: irradiance_wm2

211	function	<code>__init__</code>	args: self, csv_path, device
219	function	<code>sample</code>	args: self, x, y
237	function	<code>__init__</code>	args: self, csv_path, device
244	function	<code>sample</code>	args: self, x
253	function	<code>__init__</code>	args: self, csv_path, start_utc, device
306	function	<code>refine_time_grid</code>	args: self, max_step_sec
375	function	<code>_sample_matrix</code>	args: self, matrix, t_sec, s_km
392	function	<code>sample</code>	args: self, name, t_sec, s_km
395	function	<code>register_time_integral</code>	args: self, name, rate
405	function	<code>sample_integral</code>	args: self, name, t_sec, s_km
428	function	<code>register_soc_time_integral</code>	args: self, name, soc_grid, rate
454	function	<code>sample_soc_integral</code>	args: self, name, t_sec, s_km, soc
480	function	<code>interpolate</code>	args: values, time_index
498	function	<code>parse_args</code>	
554	function	<code>main</code>	
912	function	<code>pv_power</code>	args: t_sec, s_km
944	function	<code>register_stationary_mode</code>	args: prefix, tilt_fraction
1010	function	<code>apply_wait</code>	args: t_sec, soc, tb_c, duration, s_km
1091	function	<code>evaluate</code>	args: policy
1097	function	<code>append_wait_trace</code>	args: label, segment_index, s_km, t_before, t_after, soc_before, soc_after, tb_before, tb_after
1443	function	<code>evaluate_population</code>	args: policy

ファイルを上から読んだときの定義順

行	内容
26	例外処理を伴う try ブロックを実行する。
32	ROOT に <code>Path(__file__).resolve().parents[1]</code> の結果を代入する。
33	条件 <code>str(ROOT) not in sys.path</code> を判定し、真なら内部処理を行う。
39	関数 <code>resolve</code> を定義する。
44	関数 <code>iso_utc</code> を定義する。
50	関数 <code>source_signature</code> を定義する。
62	関数 <code>sample_cem_noise</code> を定義する。
85	関数 <code>cem_should_stop</code> を定義する。
102	関数 <code>resolve_cuda_graph_enabled</code> を定義する。
109	関数 <code>build_distance_segments</code> を定義する。
133	関数 <code>kinetic_power_w</code> を定義する。
146	関数 <code>slew_limited_segment_kinematics</code> を定義する。
193	関数 <code>stationary_auxiliary_power_w</code> を定義する。
208	クラス <code>TensorMap2D</code> を定義する。
236	クラス <code>TensorMap1D</code> を定義する。
252	クラス <code>WeatherGrid</code> を定義する。
498	関数 <code>parse_args</code> を定義する。

554 関数 `main` を定義する。
1693 条件 `__name__ == '__main__'` を判定し、真なら内部処理を行う。

import 群の詳細

行	import 文	このファイルで読む理由
10	<code>from __future_ _import annotations</code>	型ヒントの遅延評価を有効にし、自己参照型や <code>forward reference</code> を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
12	<code>import argparse</code>	CLI 引数を宣言し、実行時パラメータを外から受け取るため。このファイル内での主な使用位置は L498, L499, L509, L515, L521, L544。
13	<code>import hashlib</code>	snapshot ID や入力資産の <code>digest</code> を作るため。このファイル内での主な使用位置は L51。
14	<code>import json</code>	<code>manifest</code> 、 <code>checkpoint</code> 、UDP payload をやり取りするため。このファイル内での主な使用位置は L58, L891, L1451, L1556, L1605, L1686, L1689。
15	<code>import math</code>	<code>clip</code> 、 <code>sqrt</code> 、 <code>sin/cos</code> 、有限判定など数値ロジックに使うため。このファイル内での主な使用位置は L314, L342, L678, L1179。
16	<code>import sys</code>	<code>sys</code> モジュールを利用するため。このファイル内での主な使用位置は L33, L34。
17	<code>from datetime import datetime, timedelta, timezone</code>	UTC 時刻や相対時間を扱うため。このファイル内での主な使用位置は L44, L46, L47, L253, L734, L742, L751, L752, ...。
18	<code>from pathlib import Path</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L32, L39, L40, L50, L211, L237, L253, L500, ...。
19	<code>from zoneinfo import ZoneInfo</code>	<code>zoneinfo</code> から <code>ZoneInfo</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L691。
21	<code>import numpy as np</code>	数値配列、 <code>clip</code> 、補間、統計計算を行うため。このファイル内での主な使用位置は L113, L115, L116, L117, L120, L121, L127, L128, ...。
22	<code>import pandas as pd</code>	CSV や時刻列の読込・整形・再サンプリングに使うため。このファイル内での主な使用位置は L212, L238, L254, L255, L257, L259, L265, L267, ...。
23	<code>import torch</code>	<code>torch</code> モジュールを利用するため。このファイル内での主な使用位置は L66, L68, L75, L80, L81, L82, L135, L136, ...。
24	<code>import yaml</code>	<code>profile</code> 、 <code>stop</code> 、 <code>schedule</code> 、 <code>summary</code> YAML を読み書きするため。このファイル内での主な使用位置は L565, L578, L686。
27	<code>from torch.utils. tensorboard import SummaryWriter</code>	<code>torch.utils.tensorboard</code> から <code>SummaryWriter</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L29, L904。
36	<code>from mpc_solarcar.route_ utils import average_profile_ segments</code>	<code>route_utils.py</code> から <code>average_profile_segments</code> を読み込み、このファイルの内部処理を分担させるため。実体ファイルは <code>mpc_solarcar/route_utils.py</code> 。このファイル内での主な使用位置は L626, L629。

関数・クラスを上から順に解説

L39 関数 `resolve`

- 定義: `resolve(profile, value)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `Path`, `is_absolute`, `resolve`, `str`
 - 戻り値の要点: `path if path.is_absolute() else (profile.parent / path).resolve()`
1. `path` に `Path(str(value))` の結果を代入する。
 2. `path if path.is_absolute() else (profile.parent / path).resolve()` を返す。

L44 関数 `iso_utc`

- 定義: `iso_utc(value)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `astimezone`, `fromisoformat`, `replace`, `str`
 - 戻り値の要点: `parsed.replace(tzinfo=timezone.utc) if parsed.tzinfo is None else parsed.astimezone(timezone.utc)`
1. `text` に `str(value).replace('Z', '+00:00')` の結果を代入する。
 2. `parsed` に `datetime.fromisoformat(text)` の結果を代入する。
 3. `parsed.replace(tzinfo=timezone.utc) if parsed.tzinfo is None else parsed.astimezone(timezone.utc)` を返す。

L50 関数 `source_signature`

- 定義: `source_signature(paths, settings)`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `dumps`, `encode`, `hexdigest`, `iter`, `open`, `read`, `resolve`, `sha256`, `str`, `update`
 - 戻り値の要点: `digest.hexdigest()`
1. `digest` に `hashlib.sha256()` の結果を代入する。
 2. `paths` を順に走査し、各要素を `path` に入れて処理する。
 3. `digest.update(...)` を実行する。
 4. `digest.hexdigest()` を返す。

L62 関数 `sample_cem_noise`

- 定義: `sample_cem_noise(population, dimensions, device, antithetic)`
- docstring: Sample CEM perturbations, reserving the first row for the current mean.
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `cat`, `int`, `randn`, `zero_`, `zeros`
- 戻り値の要点: `torch.cat((torch.zeros((1, dimensions), device=device), paired), dim=0) / noise`

1. `population` に `int(population)` の結果を代入する。
2. `dimensions` に `int(dimensions)` の結果を代入する。
3. 条件 `population <= 0 or dimensions <= 0` を判定し、真なら内部処理を行う。
4. 条件 `not antithetic or population == 1` を判定し、真なら内部処理を行う。
5. `pair_count` に `(population - 1 + 1) // 2` の結果を代入する。
6. `positive` に `torch.randn((pair_count, dimensions), device=device)` の結果を代入する。
7. `paired` に `torch.cat((positive, -positive), dim=0)[:population - 1]` の結果を代入する。
8. `torch.cat((torch.zeros((1, dimensions), device=device), paired), dim=0)` を返す。

L85 関数 `cem_should_stop`

- 定義: `cem_should_stop(generation_completed, stagnant_generations, mean_std_kmh, patience, min_generations, max_std_kmh)`
 - docstring: Return true only after both objective and sampling spread have converged.
 - このブロックが直接呼ぶ主な関数・メソッド: `float`, `int`, `max`
 - 戻り値の要点: `float(max_std_kmh) <= 0.0 or float(mean_std_kmh) <= float(max_std_kmh) / False / False`
1. 条件 `int(patience) <= 0 or int(generation_completed) + 1 < max(0, int(min_generations))` を判定し、真なら内部処理を行う。
 2. 条件 `int(stagnant_generations) < int(patience)` を判定し、真なら内部処理を行う。
 3. `float(max_std_kmh) <= 0.0 or float(mean_std_kmh) <= float(max_std_kmh)` を返す。

L102 関数 `resolve_cuda_graph_enabled`

- 定義: `resolve_cuda_graph_enabled(requested, integration_ds_km)`
 - docstring: Auto-enable graphs only for the coarse rollout shape covered by the benchmark.
 - このブロックが直接呼ぶ主な関数・メソッド: `bool`, `float`
 - 戻り値の要点: `float(integration_ds_km) >= 5.0 / bool(requested)`
1. 条件 `requested is not None` を判定し、真なら内部処理を行う。
 2. `float(integration_ds_km) >= 5.0` を返す。

L109 関数 `build_distance_segments`

- 定義: `build_distance_segments(race_km, integration_ds_km, stop_distances_km)`
 - docstring: Build integration segments with every physical stop as an exact boundary.
 - このブロックが直接呼ぶ主な関数・メソッド: `arange`, `asarray`, `astype`, `concatenate`, `diff`, `unique`
 - 戻り値の要点: `(boundaries[:-1].astype(np.float32), np.diff(boundaries).astype(np.float32), boundaries)`
1. `base_boundaries` に `np.arange(0.0, race_km, integration_ds_km, dtype=np.float64)` の結果を代入する。
 2. `boundaries` に `np.unique(np.concatenate((base_boundaries, np.asarray(stop_distances_km, dtype=np.float64), np.asarray([race_km], dtype=np.float64))))` の結果を代入する。
 3. `boundaries` に `boundaries[(boundaries >= 0.0) & (boundaries <= race_km)]` の結果を代入する。
 4. `(boundaries[:-1].astype(np.float32), np.diff(boundaries).astype(np.float32), boundaries)` を返す。

L133 関数 `kinetic_power_w`

- 定義: `kinetic_power_w(mass_kg, speed_ms, previous_speed_ms, dt_sec)`
 - docstring: Signed wheel power whose interval integral equals the kinetic-energy change.
 - このブロックが直接呼ぶ主な関数・メソッド: `clamp`, `float`
 - 戻り値の要点: `delta_energy_j / torch.clamp(dt_sec, min=1e-06)`
1. `delta_energy_j` に `0.5 * float(mass_kg) * (speed_ms * speed_ms - previous_speed_ms * previous_speed_ms)` の結果を代入する。
 2. `delta_energy_j / torch.clamp(dt_sec, min=1e-06)` を返す。

L146 関数 `slew_limited_segment_kinematics`

- 定義: `slew_limited_segment_kinematics(previous_speed_ms, target_speed_ms, distance_km, accel_limit_kmhps, decel_limit_kmhps)`
 - docstring: Return distance-average speed, end speed, and time for a slew-limited segment.
 - このブロックが直接呼ぶ主な関数・メソッド: `abs`, `clamp`, `float`, `full_like`, `max`, `sqrt`, `where`, `zeros_like`
 - 戻り値の要点: `(average_speed_ms, end_speed_ms, duration_sec)`
1. `distance_m` に `max(0.0, float(distance_km) * 1000.0)` の結果を代入する。
 2. `accel_ms2` に `max(1e-06, float(accel_limit_kmhps) / 3.6)` の結果を代入する。
 3. `decel_ms2` に `max(1e-06, float(decel_limit_kmhps) / 3.6)` の結果を代入する。
 4. `accelerating` に `target_speed_ms >= previous_speed_ms` の結果を代入する。
 5. `rate_ms2` に `torch.where(accelerating, torch.full_like(target_speed_ms, accel_ms2), torch.full_like(target_speed_ms, decel_ms2))` の結果を代入する。
 6. `signed_rate_ms2` に `torch.where(accelerating, rate_ms2, -rate_ms2)` の結果を代入する。
 7. `distance_to_target_m` に `torch.abs(target_speed_ms * target_speed_ms - previous_speed_ms * previous_speed_ms) / (2.0 * rate_ms2)` の結果を代入する。

8. `reaches_target` に `distance_to_target_m <= distance_m + 1e-07` の結果を代入する。
9. `limited_end_sq` に `torch.clamp(previous_speed_ms * previous_speed_ms + 2.0 * signed_rate_ms2 * distance_m, min=0.0)` の結果を代入する。
10. `limited_end_speed_ms` に `torch.sqrt(limited_end_sq)` の結果を代入する。

L193 関数 `stationary_auxiliary_power_w`

- 定義: `stationary_auxiliary_power_w(irradiance_wm2, day_power_w, night_power_w, night_threshold_wm2)`
- docstring: Match the production model's stopped day/night auxiliary-power rule.
- このブロックが直接呼ぶ主な関数・メソッド: `float`, `full_like`, `max`, `where`
- 戻り値の要点: `torch.where(irradiance_wm2 > max(0.0, float(night_threshold_wm2)), torch.full_like(irradiance_wm2, max(0.0, float(day_power_w))), torch.full_like(irradiance_wm2, max(0.0, float(night_power_w))))`
- 1. `torch.where(irradiance_wm2 > max(0.0, float(night_threshold_wm2)), torch.full_like(irradiance_wm2, max(0.0, float(day_power_w))), torch.full_like(irradiance_wm2, max(0.0, float(night_power_w))))` を返す。

L208 クラス `TensorMap2D`

- 定義: `TensorMap2D(bases=ⓂⓂ)`
- docstring: Bilinear CSV-map interpolation that stays on the selected torch device.
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `as_tensor`, `clamp`, `contiguous`, `len`, `maximum`, `minimum`, `read_csv`, `searchsorted`, `to_numpy`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 1. docstring または説明文字列を置く。
- 2. 関数 `__init__` を定義する。
- 3. 関数 `sample` を定義する。

L236 クラス `TensorMap1D`

- 定義: `TensorMap1D(bases=ⓂⓂ)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `ValueError`, `as_tensor`, `clamp`, `contiguous`, `len`, `maximum`, `minimum`, `read_csv`, `searchsorted`, `to_numpy`
- 戻り値の要点: 戻り値が無い、`return` 文が明示されていない。
- 1. 関数 `__init__` を定義する。
- 2. 関数 `sample` を定義する。

L252 クラス WeatherGrid

- 定義: WeatherGrid(bases=`object`)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: Index, RuntimeError, Series, Timestamp, ValueError, _sample_matrix, any, append, arange, array, as_tensor, asarray
- 戻り値の要点: 戻り値が無いか、return 文が明示されていない。

1. 関数 __init__ を定義する。
2. 関数 refine_time_grid を定義する。
3. 関数 _sample_matrix を定義する。
4. 関数 sample を定義する。
5. 関数 register_time_integral を定義する。
6. 関数 sample_integral を定義する。
7. 関数 register_soc_time_integral を定義する。
8. 関数 sample_soc_integral を定義する。

L498 関数 parse_args

- 定義: parse_args()
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: ArgumentParser, add_argument, parse_args
- 戻り値の要点: parser.parse_args()

1. parser に argparse.ArgumentParser(description=__doc__) の結果を代入する。
2. parser.add_argument(...) を実行する。
3. parser.add_argument(...) を実行する。
4. parser.add_argument(...) を実行する。
5. parser.add_argument(...) を実行する。
6. parser.add_argument(...) を実行する。
7. parser.add_argument(...) を実行する。
8. parser.add_argument(...) を実行する。
9. parser.add_argument(...) を実行する。
10. parser.add_argument(...) を実行する。

L554 関数 main

- 定義: `main()`
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: `CUDAGraph`, `DataFrame`, `Path`, `RuntimeError`, `Stream`, `SummaryWriter`, `TensorMap1D`, `TensorMap2D`, `ValueError`, `WeatherGrid`, `ZoneInfo`, `abs`
 - 戻り値の要点: `0 / torch.clamp(pv, max=pv_power_limit_w) if pv_power_limit_w > 0.0 else pv / (t_sec + duration, soc, tb_c) / (cost, soc, min_soc_seen, t_sec, max_current_seen, max_charge_current_seen, max_timing_violation_sec, deadline_violation_sec, trace_rows)`
1. `args` に `parse_args()` の結果を代入する。
 2. `args.cuda_graph` に `resolve_cuda_graph_enabled(args.cuda_graph, args.integration_ds_km)` の結果を代入する。
 3. 条件 `not torch.cuda.is_available()` を判定し、真なら内部処理を行う。
 4. `device` に `torch.device('cuda')` の結果を代入する。
 5. `torch.manual_seed(...)` を実行する。
 6. `profile` に `args.profile.resolve()` の結果を代入する。
 7. `cfg` に `yaml.safe_load(profile.read_text(encoding='utf-8'))` or `{}` の結果を代入する。
 8. `model` に `cfg['model']` の結果を代入する。
 9. `mpc` に `cfg['mpc']` の結果を代入する。
 10. `paths` に `cfg['paths']` の結果を代入する。

CLI 引数

行	引数
500	<code>--profile</code>
501	<code>--output-dir</code>
502	<code>--integration-ds-km</code>
503	<code>--control-ds-km</code>
504	<code>--population</code>
505	<code>--elite</code>
506	<code>--generations</code>
507	<code>--seed</code>
508	<code>--checkpoint-every</code>
509	<code>--resume</code>
510	<code>--tensorboard-dir</code>
511	<code>--initial-policy</code>
512	<code>--initial-std-kmh</code>
513	<code>--antithetic-sampling</code>
519	<code>--cuda-graph</code>
528	<code>--early-stop-patience</code>
534	<code>--early-stop-min-generations</code>
535	<code>--early-stop-min-delta</code>

```
536      --early-stop-max-std-kmh  
542      --capture-final-surrogate-trace
```

処理の流れ

1. CLI 引数を解釈し、main() から処理を起動する。

このファイルの位置づけ

この資料群では、`scripts/gpu_upper_policy_search.py` を **planning research** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。